

SPECIFICA TECNICA: Servizio di Messaggistica Client-Server

Progetto: Servizio di Messaggistica Client-Server

Autore: Vittorio Iozzia 0339495



1. Introduzione: Architettura e Scelte Progettuali Chiave

Questo documento costituisce la specifica tecnica completa e l'analisi di progettazione per un'applicazione di messaggistica basata su un'architettura Client-Server. Sviluppato interamente in linguaggio C per sistemi operativi POSIX, il progetto si fonda su un insieme di scelte architettureali mirate a garantire robustezza, efficienza e isolamento del servizio. L'analisi che segue non si limiterà a descrivere il funzionamento del sistema, ma esplorerà il "perché" dietro ogni decisione implementativa.

Il progetto "poggia" su tre pilastri logici fondamentali:



Gestione della Concorrenza tramite Modello "Thread-per-Client":

Per servire un numero potenzialmente elevato di client simultaneamente è stato adottato un modello dinamico in cui ogni nuova connessione accettata genera un thread dedicato. Questa scelta garantisce la massima isolamento: un'operazione bloccante o un errore di un client non impatta sulla reattività degli altri. L'uso di `pthread_detach` assicura che le risorse del thread vengano rilasciate automaticamente alla disconnessione, prevenendo memory leak. Per prevenire attacchi di tipo *Denial of Service* (DoS) o l'esaurimento delle risorse, è stato introdotto un limite massimo di connessioni concorrenti (`MAX_CLIENTS`).



Protocollo di Comunicazione Line-Based e Thread-Safe:

La comunicazione tra client e server è governata da un protocollo semplice e ispezionabile, basato su righe di testo terminate da `\n` e campi delimitati dal carattere pipe `|`. Per superare la natura di "stream" di TCP, è stata implementata la funzione `recv_line`. Essa garantisce la ricezione atomica ed è stata potenziata per scartare stringhe in eccesso, prevenendo la desincronizzazione del protocollo in caso di buffer overflow. La successiva tokenizzazione avviene tramite `strtok_r`, assicurando la totale *reentrancy* richiesta dal multithreading.



Persistenza della Memoria con Strategia Ibrida Atomica:

Per massimizzare le performance, i dati utente usano una politica Write-Through (protetti da `file_mutex`), mentre i messaggi risiedono in una cache in RAM (`cache_mutex`). La persistenza dei messaggi è delegata a un Flush Thread asincrono in background, ottimizzato da un Dirty Bit (il salvataggio scatta solo in caso di reali modifiche). Ogni scrittura su disco è blindata dal pattern di Aggiornamento Atomico (`rename + fsync`) per prevenire corruzioni hardware, con un salvataggio di sicurezza finale forzato durante il Graceful Shutdown.

2. Specifiche Funzionali Chiave

Gestione Utenti:

Il sistema permette a nuovi utenti di registrarsi con un nome utente e una password (persistenza immediata su disco). Gli utenti esistenti possono accedere al sistema fornendo le proprie credenziali. Per arginare attacchi Brute Force, il server disconnette forzatamente i client dopo tre tentativi di login falliti consecutivi.

Concorrenza Sicura:

Il server è in grado di gestire le richieste di più client contemporaneamente. L'accesso alle risorse è gestito da livelli di mutua esclusione distinti: `cache_mutex` per l'accesso in RAM e `file_mutex` per prevenire race condition sul file system.

Robustezza e Liveness:

Il sistema implementa meccanismi per gestire la disconnessione anomala dei client tramite l'impostazione di Timeout a livello di socket (`SO_RCVTIMEO`). È garantita una chiusura pulita (Graceful Shutdown) in risposta a segnali esterni (`SIGINT`), salvando l'intero stato su disco.

3. Manuale d'Uso

Compilazione ed Esecuzione:

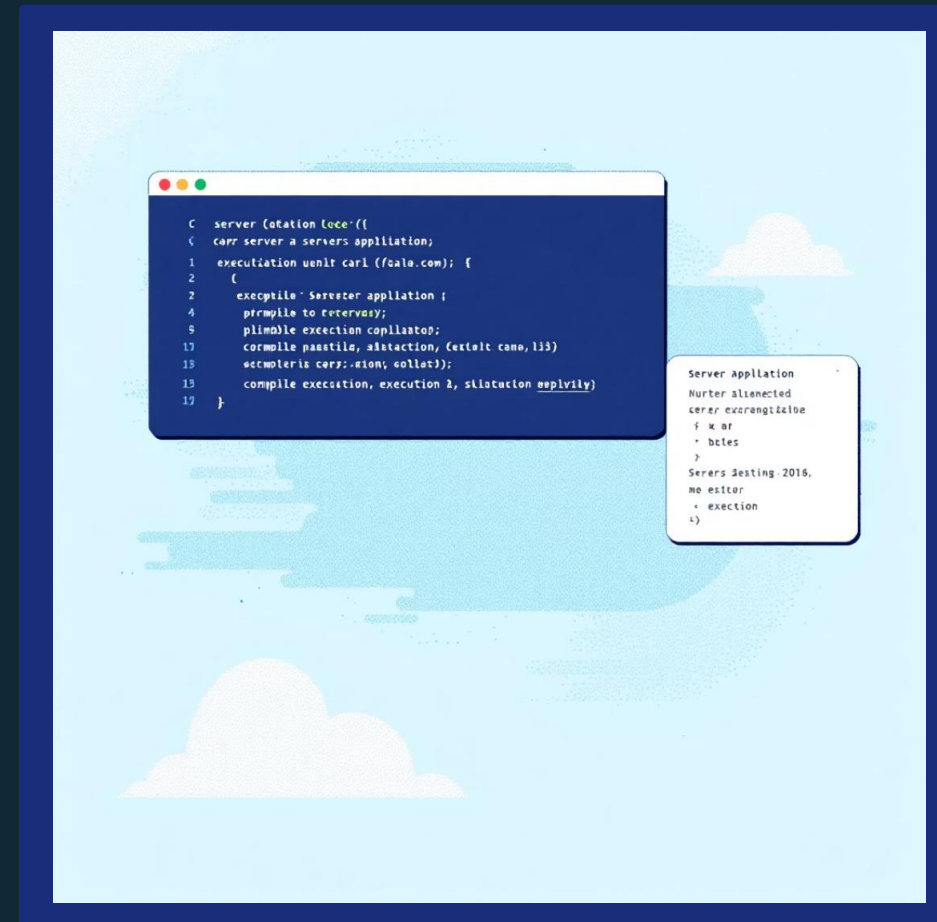
Per compilare il progetto POSIX eseguire `make posix`, per pulire gli eseguibili `make clean`. Avviare il server tramite `./server_posix`; esso si metterà in ascolto sulla porta designata e gestirà autonomamente i file `users.txt` e `messages.txt`.

Utilizzo del Client:

Avviare il client con `./client_posix`. Il menu iniziale consente di Registrarsi , Accedere o Uscire. Dopo l'autenticazione, è possibile Inviare messaggi, Leggerli per ricevere tutti i messaggi destinati al proprio utente, oppure Cancellarli.

Chiusura del Server:

Per chiudere il server in modo pulito premere Ctrl+C (SIGINT). L'intercettazione del segnale avvia una terminazione che libera le porte di rete, chiude i thread attivi e innesca il salvataggio dei messaggi pendenti tramite `atexit`.



4. Gestione dello Stato e Durabilità dei Dati

La permanenza dei dati è una caratteristica fondamentale per un sistema di messaggistica. Il progetto utilizza file di testo per garantire semplicità, trasparenza e portabilità.

File degli Utenti (users.txt):

Memorizza le credenziali (username:hash). Le operazioni sono protette dal `file_mutex`.

File dei Messaggi (messages.txt):

Rappresenta la copia persistente dello stato della cache. Il formato testuale prevede campi per destinatario, mittente, oggetto e corpo.

Caricamento e Salvataggio (Write-Back):

All'avvio, `load_messages_from_file()` popola la RAM parlando il file riga per riga. Al momento dello spegnimento, la funzione:

`(server_shutdown_cleanup)` si occupa di trasferire fisicamente la RAM su disco. Per proteggere i dati dalla corruzione per mancanza di alimentazione improvvisa, il salvataggio avviene su un file `.tmp`, i dati vengono consolidati nel disco fisico bypassando la cache dell'OS (`fsync`) e, solo al termine, il file sostituisce l'originale con l'operazione atomica `rename()`.

5. Standardizzazione e Protocollo Client-Server

In un'applicazione Client-Server è fondamentale che entrambe le parti concordino sulle dimensioni massime dei dati e sulle regole del protocollo di comunicazione.



Centralizzazione (common.h):

Tutte le configurazioni (es. `MAX_USERNAME_LEN`, `MAX_MSG_LEN`) e le macro di comando/risposta (es. `CMD_LOGIN`, `RESP_OK`) sono definite in `common.h`. Questo elimina l'uso di stringhe magiche hardcoded e assicura che client e server abbiano una visione identica dei limiti dimensionali, eliminando incoerenze e buffer overflow.



Parsing Dinamico e Sicuro:

La validazione dell'input dell'utente non si affida a formati statici. Le funzioni `sscanf` costruiscono i propri formati di lettura dinamicamente tramite `snprintf` interfacciandosi con le macro di `common.h`. Inoltre, tutte le conversioni stringa-intero utilizzano `strtol` analizzando esplicitamente il puntatore di fine conversione per prevenire comportamenti indefiniti.



Il Protocollo Testuale:

Il sistema adotta un protocollo line-based (es. `SEND|recipient|subject|body`) per semplicità di implementazione e trasparenza durante il debugging. La `recv_line` dedicata assicura il riassemblaggio della logica applicativa indipendentemente dalla frammentazione dei pacchetti TCP.

6. Autenticazione e Prevenzione Race Condition

Le funzionalità di autenticazione sono gestite dalle funzioni `create_user` e `authenticate`, interagenti col client che si occupa della raccolta e sanitizzazione preliminare delle credenziali.

Flusso Operativo e Sincronizzazione Atomica: Quando un client invia il comando REGISTER, il server processa il payload nel thread dedicato. Per scongiurare attacchi o inconsistenze derivanti dalla vulnerabilità *Time-of-Check to Time-of-Use* (TOCTOU), la funzione `create_user` esegue sia la verifica di pre-esistenza dell'utente sia l'apertura e scrittura del file all'interno della medesima ed ininterrotta sezione critica (lock su `file_mutex`). In caso di esito positivo o negativo, il server risponde utilizzando rigorosamente le risposte macro predefinite e non messaggi testuali arbitrari, consentendo al client un riconoscimento e parsing esatto.

7. Sicurezza dei Dati (Hashing e Buffer)

La sicurezza rappresenta un aspetto rilevante per la gestione delle credenziali. Il sistema non memorizza mai le password in chiaro sul disco, ma utilizza una funzione di hash. La funzione `hash_password` utilizza l'algoritmo djb2. Sebbene non crittografico, garantisce determinismo ed efficienza per i requisiti didattici del progetto, mitigando l'esposizione diretta in caso di compromissione del file `users.txt`. Dal lato software, ogni input scambiato via socket è soggetto a sanitizzazione: i caratteri speciali delimitatori del protocollo (`|` e `\n`) vengono sovrascritti qualora iniettati impropriamente nei campi, annullando il rischio di *Protocol Injection* o alterazioni del parsing.

8. Gestione dei Segnali e dello Spegnimento

Il progetto implementa una gestione dei segnali semplice ma efficace per garantire una chiusura pulita (graceful shutdown).

- **Server (SIGINT e Sblocco Accept):** Quando il segnale SIGINT viene ricevuto dall'handler, esso non si limita ad impostare la variabile globale atomica `running = 0`. Per risolvere il problema del blocco deterministico sulla system call `accept()`, l'handler invoca `shutdown(server_fd, SHUT_RDWR)`. Questo interrompe forzatamente la comunicazione di rete, risvegliando il thread principale e consentendo al processo di concludersi ordinatamente invocando le `atexit` per la persistenza finale.
- **Client (Gestione Granulare del SIGPIPE):** Piuttosto che ignorare globalmente il segnale SIGPIPE con `signal(SIGPIPE, SIG_IGN)`, il progetto utilizza il flag di sistema `MSG_NOSIGNAL` nelle funzioni di invio socket. Questo approccio più robusto impedisce al processo di crashare per disconnessioni remote, convertendo il segnale fatale in un normale errore EPIPE, permettendo al thread di liberare la memoria in autonomia.

9. Gestione delle Disconnessioni e Liveness

La gestione corretta delle disconnessioni è fondamentale per la robustezza di un server concorrente a lunga esecuzione. Ogni client è gestito da un thread la cui vita è legata al ciclo del polling sulla rete. Il rilevamento della disconnessione avviene tramite i valori di ritorno (0 o -1) di `recv_line`. Per evitare che client malevoli o soggetti a caduta di rete generino *thread zombies* o blocchino i socket in attesa infinita, sono stati implementati timeout a livello kernel configurando le opzioni del socket `SO_RCVTIMEO` e `SO_SNDTIMEO`. Una volta interrotto il ciclo, il socket viene chiuso tramite `close` per liberare il file descriptor e la memoria `ClientHandler` viene deallocata per prevenire leak. Essendo creati in modalità detached, non vi è necessità di espliciti `pthread_join`.

10. Interfaccia Applicativa e Cancellazione Efficiente

Le funzionalità di bacheca sono processate in memoria, assicurando una velocità di recupero dati massima senza I/O bloccante.

- **READ (Ricezione Multilinea):** Per superare il problema del protocollo asincrono, il server risponde a una query di lettura premettendo un header formattato (`RESP_COUNT | numero_messaggi`). Il client, estraendo questo valore con controllo dinamico, entra in un ciclo definito che impedisce attese indefinite o sfasamenti dei pacchetti TCP.
- **DELETE (Sicurezza e Compattazione):** La cancellazione dei messaggi è implicitamente autorizzata dal server (il client non ha controllo sul layer di autorizzazione). Quando un utente cancella i propri messaggi, la struttura dinamica esegue una compattazione in-place con complessità temporale $O(N)$ e spaziale $O(1)$. L'algoritmo scorre l'array una sola volta, scartando gli elementi target e compattando i restanti tramite un indice di scrittura indipendente.

11. Gestione Stream TCP (Prevenzione Desincronizzazione)

TCP garantisce che i dati arrivino nell'ordine corretto, ma non con chiamate atomiche. Per evitare che un comando parziale corrompa lo stream, la logica applicativa è gestita rigorosamente da `recv_line` e `send_all`.

- **Recv_line:** Accumula byte ignorando selettivamente `\r` e terminando in corrispondenza del `\n`. Fondamentale per la sicurezza, se l'input supera le dimensioni del buffer, la funzione termina in modo sicuro la stringa allocata ma continua a prelevare e scartare dal socket i byte in eccesso fino al newline. Questo impedisce catastroficamente che le code dei comandi troppo lunghi vengano interpretati come intestazioni nel ciclo successivo.
- **Send_all:** Gestisce il fenomeno delle scritture parziali su stream TCP garantendo che l'intero buffer venga spedito iterativamente utilizzando l'aritmetica dei puntatori per calcolare l'offset dei byte residui ad ogni ciclo kernel.

12. Future Works e Conclusioni

Il progetto offre una base estremamente robusta con vari spunti di estensione futura. Evoluzioni naturali comprendono la suddivisione dell'archivio messaggi su più file basati su intervalli temporali , meccanismi di lazy loading o caching selettivo per ottimizzare la località dei dati , e l'adozione di un'architettura a *Thread Pool* in luogo del Thread-per-Client. Inoltre, in un contesto di produzione, l'algoritmo di hashing djb2 verrebbe rimpiazzato con algoritmi crittografici lenti (es. Argon2) e lo scambio dati cifrato via TLS.

In conclusione, è stato descritto un servizio di messaggistica implementato per sistemi POSIX. Il progetto dimostra l'applicazione coerente dei concetti fondamentali dei sistemi operativi: gestione concorrente sicura contro le race-condition , protocolli line-based atomici , persistenza con durabilità dei dati (fsync) , gestione avanzata dei segnali per Graceful Shutdown e protezioni difensive contro attacchi (DoS, injection, buffer overflow). Gli obiettivi didattici e professionali prefissati sono stati raggiunti con assoluto successo.